

Projeto Final

Estrutura de Dados e Complexidade de Algoritmos

TSP com Heurística e VND

Joaquim Breno Brito Cavalcante
Universidade Federal da Paraíba
Centro de Informática
Prof. Gilberto Farias

Junho de 2026

1 Definição do POC

Escolhi o Problema do Caixeiro Viajante, o famoso TSP.

A ideia é simples, tem um monte de cidades e o caixeiro precisa visitar todas, uma vez cada, e voltar pra onde começou. O objetivo é gastar o mínimo de distância (ou custo) possível no caminho todo.

Dá pra pensar num grafo completo onde cada par de cidades tem um custo de ir de uma pra outra, geralmente a distância. A solução é só a ordem em que as cidades são visitadas, como ir de A pra C, depois B, depois D, e fechar o ciclo.

Regras:

- passa em cada cidade exatamente uma vez;
- no final volta pra cidade inicial.

2 Prova de pertencimento a NP

Na aula de NP-completude a gente viu que primeiro convém olhar a versão de decisão do problema.

No TSP-DEC a pergunta é: dado os custos entre cidades e um número k , existe alguma rota que visita tudo e custa no máximo k ?

Certificado: a lista com a ordem das cidades. Tipo: 3, 1, 5, 2, 4.

Como verificar: alguém pega essa lista e:

1. vê se todas as cidades aparecem uma vez só, sem repetir e sem faltar ninguém.
2. soma o custo de cada trecho, incluindo a volta pro início.
3. responde sim se der menor ou igual a k , senão responde não.

Isso dá pra fazer em tempo linear no número de cidades. Então o TSP-DEC está em NP.

3 NP-difícil e redução

Pra mostrar que é difícil de verdade, reduz o Ciclo Hamiltoniano (CH) pro TSP-DEC. CH é aquele problema clássico: existe um ciclo que passa por todos os vértices do grafo uma vez? Isso é NP-completo, apareceu nos slides da disciplina.

Como montar a redução

Pega a instância do CH e transforma assim:

- mesmas cidades (vértices);
- se existe aresta entre duas cidades, o custo vira 1;
- se não existe aresta, o custo vira 2;
- k fica igual ao total de cidades.

Montar a matriz de custos leva tempo quadrático no número de cidades, o que ainda é polinomial.

Por que funciona

Se o grafo original tem ciclo hamiltoniano, dá pra montar um tour no TSP só com arestas de custo 1. Aí o total fica igual ao número de cidades, e isso passa no limite k.

Se não tem ciclo hamiltoniano, qualquer tour completo vai ser obrigado a usar pelo menos uma aresta que não existia no grafo original, ou seja, custo 2. O melhor caso vira número de cidades mais 1, que é maior que k.

Com isso CH reduz polinomialmente pro TSP-DEC. CH é NP-completo, então TSP-DEC é NP-difícil. E como TSP-DEC também está em NP, ele é NP-completo. A versão de otimização (achar o menor custo) herda a mesma dificuldade.

Ciclo Hamiltoniano $\xrightarrow{\text{custo 1 se tem aresta}}$ TSP-DEC $\longrightarrow$ TSP otimização

4 Instâncias da literatura

Usei instâncias do TSPLIB, que é o repositório clássico de benchmark pra TSP: <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>

Instância	Cidades	Ótimo	Observação
eil51	51	426	pequena, clássica
berlin52	52	7542	coordenadas euclidianas
st70	70	675	tamanho médio
kroA100	100	21282	a maior que testei

Tabela 1: Instâncias TSPLIB usadas nos testes.

5 Implementação e resultados (A2)

Código no GitHub: https://github.com/JoaquimBreno/atividades_modulo3_estrutura

5.1 Representação da solução

Guardei a solução como uma lista de inteiros, cada um é uma cidade na ordem da visita. Exemplo: [0, 3, 1, 4, 2].

O custo é a soma das distâncias de cada par consecutivo, mais o trecho de volta da última cidade pra primeira.

Na leitura do arquivo .tsp eu monto a matriz de distâncias inteira e deixo na memória, assim não fico recalculando coordenada o tempo todo.

5.2 Heurística de construção: vizinho mais próximo

Começa numa cidade (sorteada aleatoriamente nos testes) e, enquanto sobrar cidade, vai sempre pra mais perto que ainda não visitou. Quando acaba, o ciclo fecha sozinho na hora de calcular o custo.

Roda em tempo quadrático no número de cidades. Simples e rápido de implementar, como vi na aula.

5.3 Movimentos de vizinhança

Usei três vizinhanças no VND:

- 2-opt: inverte um pedaço da rota;
- swap: troca duas cidades de posição;
- relocate: tira uma cidade de um lugar e encaixa em outro.

5.4 VND

O VND pega a solução do vizinho mais próximo e tenta melhorar. Passa pelas vizinhanças nessa ordem: 2-opt, swap, relocate.

Se alguma melhora, volta pro 2-opt. Se nenhuma das três consegue melhorar, para. Ótimo local, sem frescura.

5.5 Resultados computacionais

Rodei cada instância 10 vezes com cidade inicial aleatória. A tabela mostra o melhor custo que apareceu, o ótimo conhecido da literatura, o gap em por cento e o tempo médio.

Instância	Melhor	Ótimo	Gap (%)	Tempo médio (s)
berlin52	7542	7542	0.0	0.035
eil51	429	426	0.7	0.047
kroA100	21379	21282	0.5	0.362
st70	683	675	1.2	0.151

Tabela 2: Resultados do VND nas instâncias TSPLIB.

Os gaps ficaram abaixo de 1,5% em todos os casos. Pra uma heurística simples, sem tabu search nem simulated annealing, acho que tá ok. O berlin52 chegou no ótimo em uma das execuções. O tempo continua baixo, frações de segundo até com 100 cidades.